System Requirements

Improved Library Discovery and Management System for WG Solheim Library

Group ACF

Renson Nicolai Villareal

John Ray Donovan Añora

Ronaldvince Yrlwin Aquino

Course: CS 191 – WFQ1: Software Engineering I

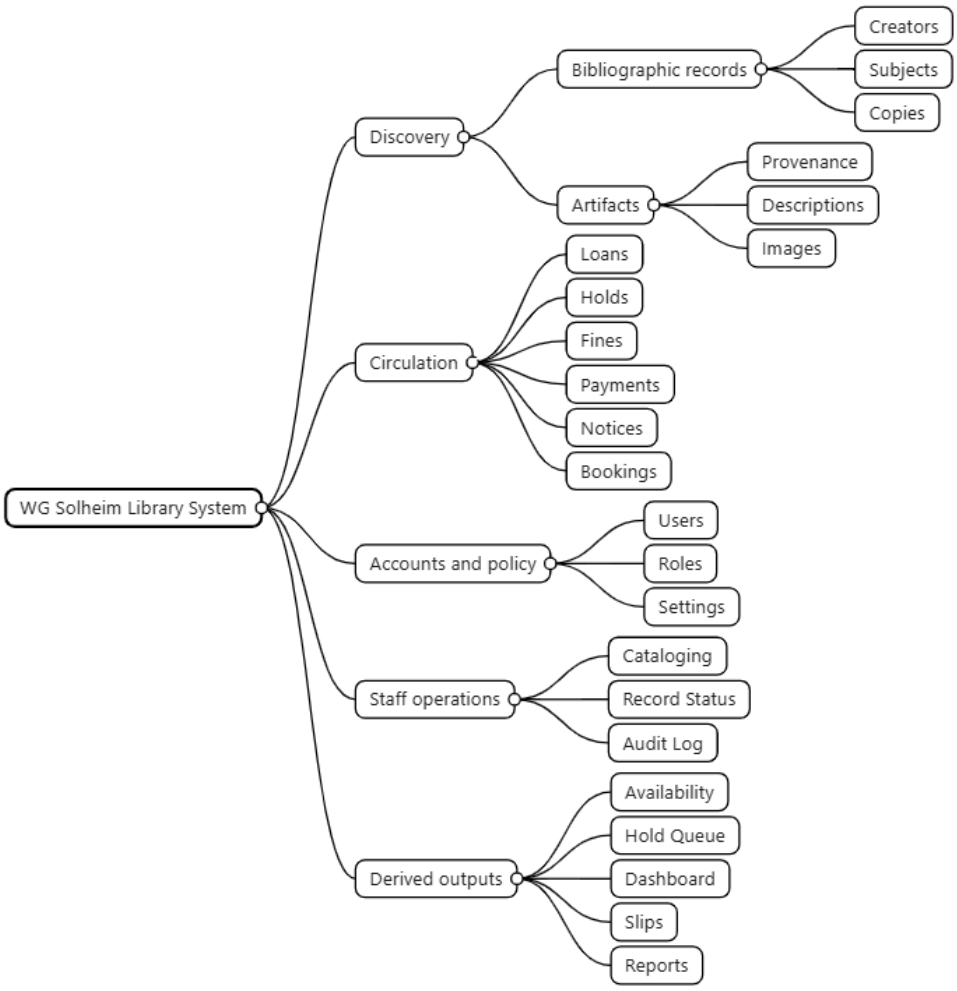
Instructor: Ligaya Leah Figueroa

Department: Department of Computer Science, University of the Philippines Diliman

Client: WG Solheim Library

Project Website: [TBA]

Content Model



Actors and Goals

***Library Users*.** Students, faculty, and researchers whose primary goal is to efficiently search, browse, and access library resources and artifact information.

Library Staff. Library personnel whose goal is to manage catalog records at their own pace in a private database, process borrowing workflows, and batch export/import data to the centralized TUKLAS system.

Use Cases

Library User Use Cases

Search Resources. A user enters search terms to locate physical books, electronic resources, or artifacts.

Filter Search Results. A user applies specific filters (e.g., author, subject, availability) to narrow down a large set of search results.

View Resource Details A user selects a specific item from the search results to view its publication information, call number, location, and availability.

View Artifact Details. If the resource is an archaeological artifact, the user views high-resolution artifact images and rich descriptions.

Library Staff Use Cases

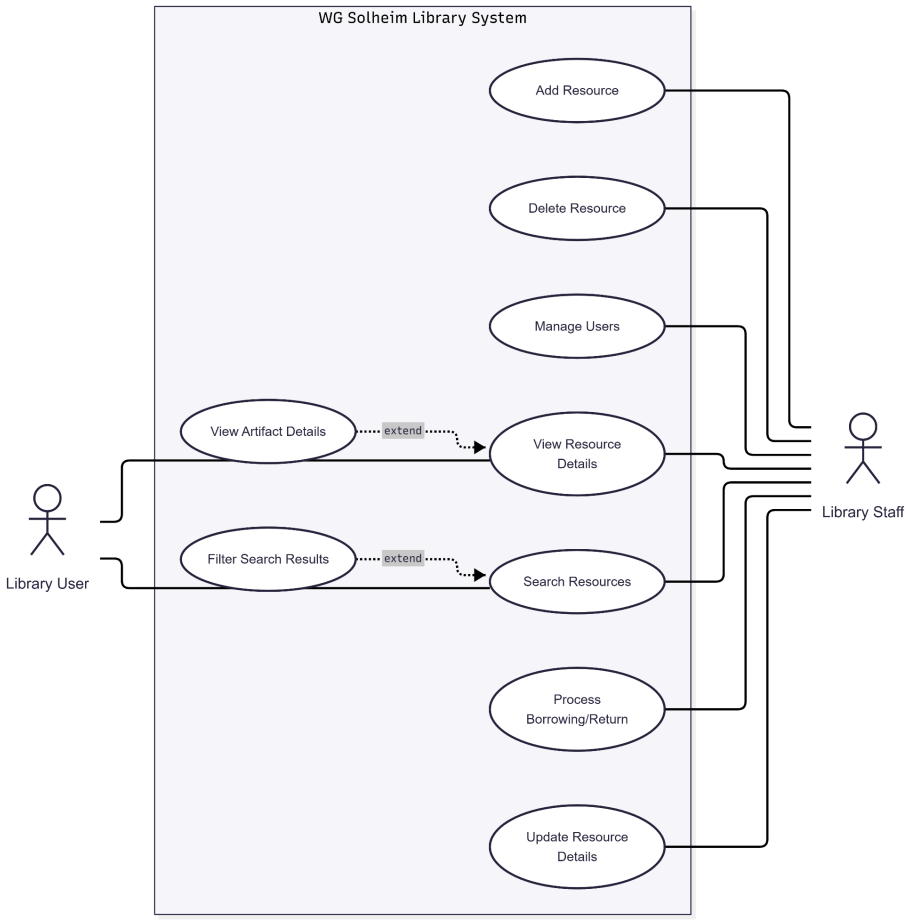
Add Resource. Library staff inputs new catalog records (books, journals, or artifacts) into the standalone database.

Update Resource Details. Library staff edits existing catalog metadata or uploads new artifact images without operational time pressure.

Delete Resource. Library staff removes an outdated or lost resource from the private catalog.

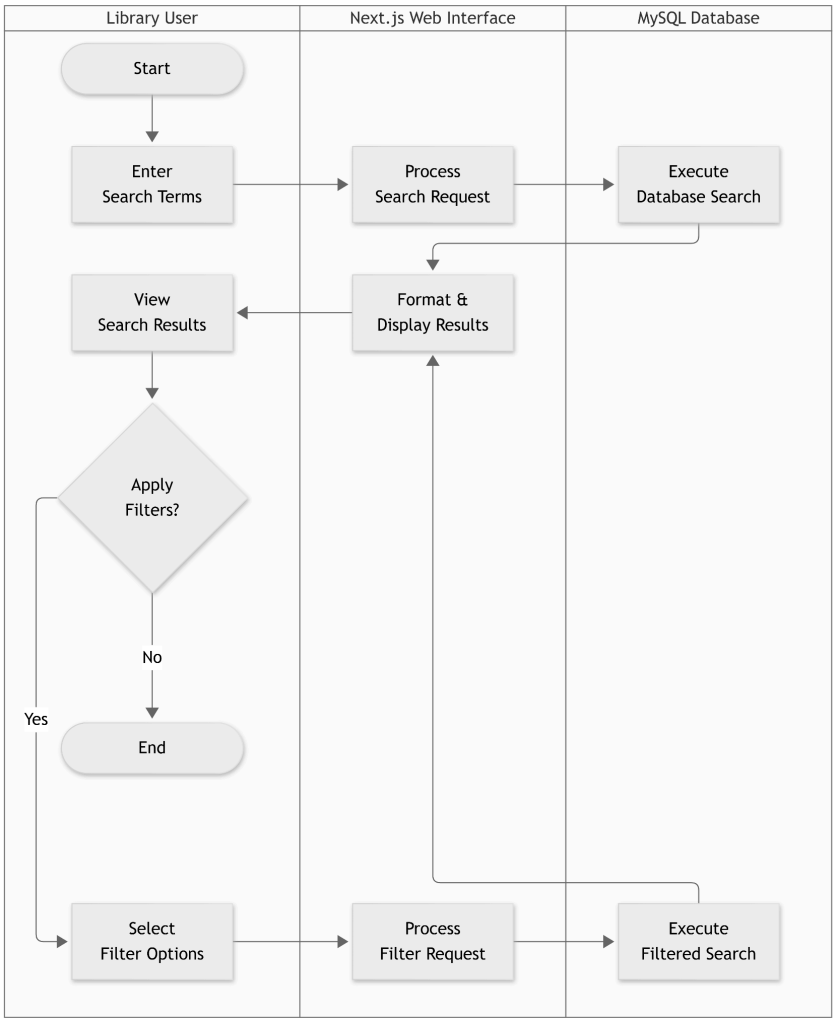
Process Borrowing/Return. Library staff logs the checkout or return of library items using the customized borrowing database system.

Manage Users. Authorized administrators add, edit, or remove user and staff accounts.

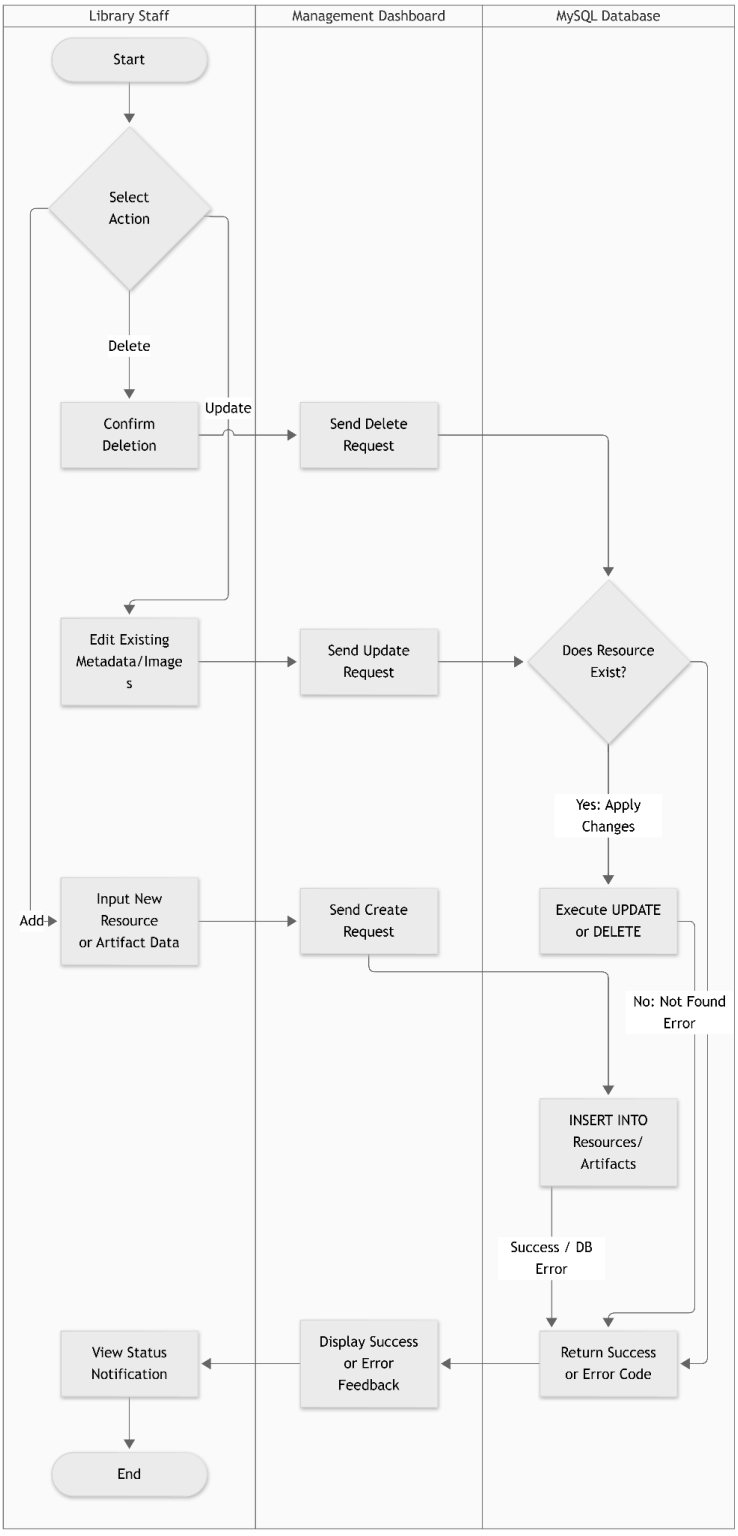


Use-case diagram

System Sequence Diagrams

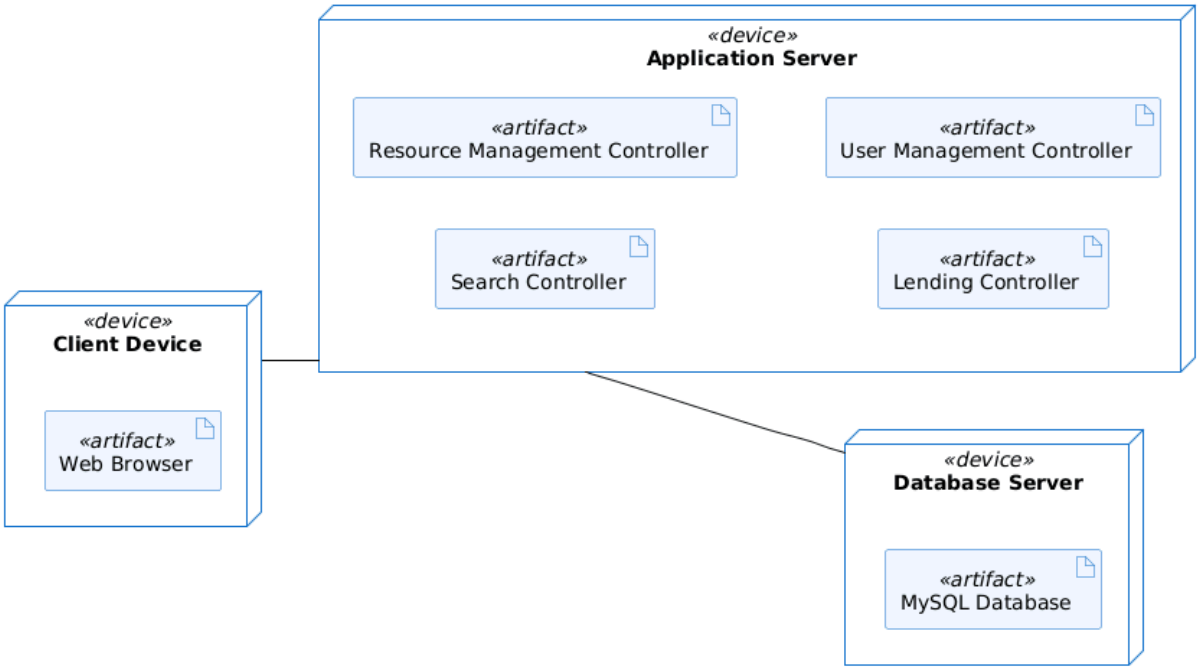


Search Resources Swimlane Diagram



Resources Management Swimlane Diagram

Configuration Model



UML Deployment Diagram

Architectural Design

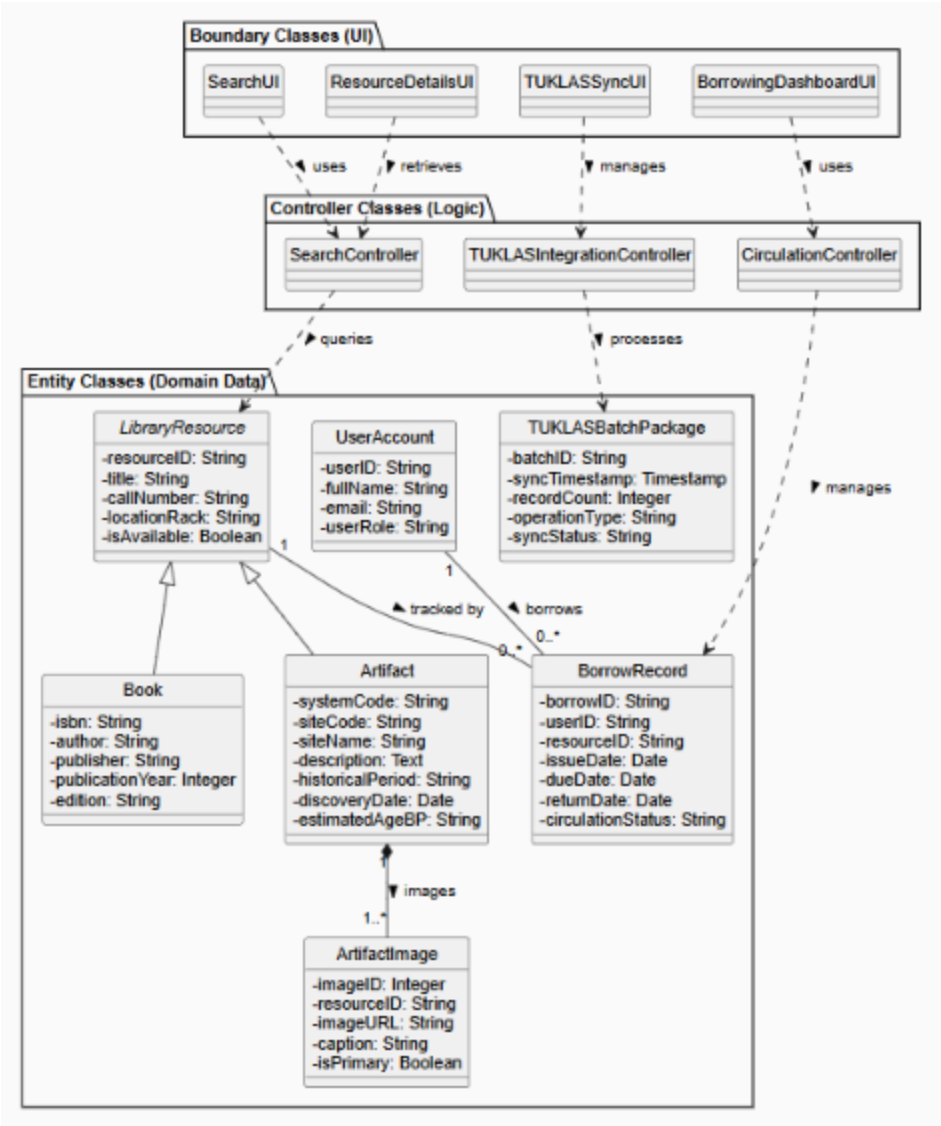
Architecture: UP Talaan combines a hierarchical content structure (home → collections → records), grid-like artifact filters, and linear staff workflows. The application uses layered MVC client-server architecture: browser/view, controller, domain logic, and MySQL data access. The PHP/MySQL stack is a draft assumption.

Quality Requirements Tree Check

Attribute	Design response
Usability	Clear public/staff navigation, help, consistent labels
Functionality	Search, facets, records, holds, circulation, reports
Reliability	Input validation, actionable errors, database constraints
Efficiency	Fast text-first pages; images load after content
Maintainability	Separate layers; shared styles, labels, and templates

Class Diagrams

The content model for the WG Solheim Library Management & Discovery System separates user interface boundaries, domain entity structures, and internal processing logic following Pressman's class design guidelines.



UML Class Diagram

Entity-Relationship Diagrams (ERDs) & Database Schema

The database model uses a private relational database schema to support independent local updates before exporting to TUKLAS.

```sql
CREATE DATABASE IF NOT EXISTS `wg_solheim_library`
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE `wg_solheim_library`;
```

```sql
-- ------------------------------------------------------------------------------
-- 1. Core resource table
-- ------------------------------------------------------------------------------

CREATE TABLE IF NOT EXISTS `RESOURCES` (
    `resource_id` VARCHAR(50) NOT NULL,
    `title` VARCHAR(255) NOT NULL,
    `call_number` VARCHAR(100) NOT NULL,
    `location_rack` VARCHAR(100) NOT NULL,
    `resource_type` VARCHAR(20) NOT NULL,
    `is_available` BOOLEAN NOT NULL DEFAULT TRUE,
    `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT `pk_resources` PRIMARY KEY (`resource_id`),
    CONSTRAINT `chk_resources_type`
        CHECK (`resource_type` IN ('BOOK', 'ARTIFACT'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------------
-- 2. Book details
-- Each book is also a resource. resource_id is both the PK and an FK.
-- ------------------------------------------------------------------------------

CREATE TABLE IF NOT EXISTS `BOOKS` (
    `resource_id` VARCHAR(50) NOT NULL,
    `isbn` VARCHAR(20) NULL,
    `author` VARCHAR(255) NOT NULL,
    `publisher` VARCHAR(150) NULL,
    `publication_year` INT NULL,
    `edition` VARCHAR(50) NULL,

    CONSTRAINT `pk_books` PRIMARY KEY (`resource_id`),
    CONSTRAINT `uq_books_isbn` UNIQUE (`isbn`),
    CONSTRAINT `fk_books_resource`
        FOREIGN KEY (`resource_id`)
        REFERENCES `RESOURCES` (`resource_id`)
        ON DELETE CASCADE
        ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------------
-- 3. MARC21 tags for books
-- ------------------------------------------------------------------------------
```

```sql
CREATE TABLE IF NOT EXISTS `BOOK_MARC_TAGS` (
    `marc_id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    `resource_id` VARCHAR(50) NOT NULL,
    `tag_number` CHAR(3) NOT NULL,
    `ind1` CHAR(1) NOT NULL DEFAULT ' ',
    `ind2` CHAR(1) NOT NULL DEFAULT ' ',
    `subfield_data` TEXT NOT NULL,

    CONSTRAINT `pk_book_marc_tags` PRIMARY KEY (`marc_id`),
    CONSTRAINT `fk_marc_tags_book`
        FOREIGN KEY (`resource_id`)
        REFERENCES `BOOKS` (`resource_id`)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    INDEX `idx_marc_tags_resource` (`resource_id`),
    INDEX `idx_marc_tags_number` (`tag_number`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ---------------------------------------------------------------------------
-- 4. Archaeological artifact details
-- ---------------------------------------------------------------------------

CREATE TABLE IF NOT EXISTS `ARTIFACTS` (
    `resource_id` VARCHAR(50) NOT NULL,
    `system_code` VARCHAR(100) NOT NULL,
    `site_code` VARCHAR(100) NOT NULL,
    `site_name` VARCHAR(255) NOT NULL,
    `description` TEXT NOT NULL,
    `historical_period` VARCHAR(100) NOT NULL,
    `discovery_date` DATE NULL,
    `estimated_age_bp` VARCHAR(100) NULL,

    CONSTRAINT `pk_artifacts` PRIMARY KEY (`resource_id`),
    CONSTRAINT `uq_artifacts_system_code` UNIQUE (`system_code`),
    CONSTRAINT `fk_artifacts_resource`
        FOREIGN KEY (`resource_id`)
        REFERENCES `RESOURCES` (`resource_id`)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    INDEX `idx_artifacts_site_code` (`site_code`),
    INDEX `idx_artifacts_period` (`historical_period`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

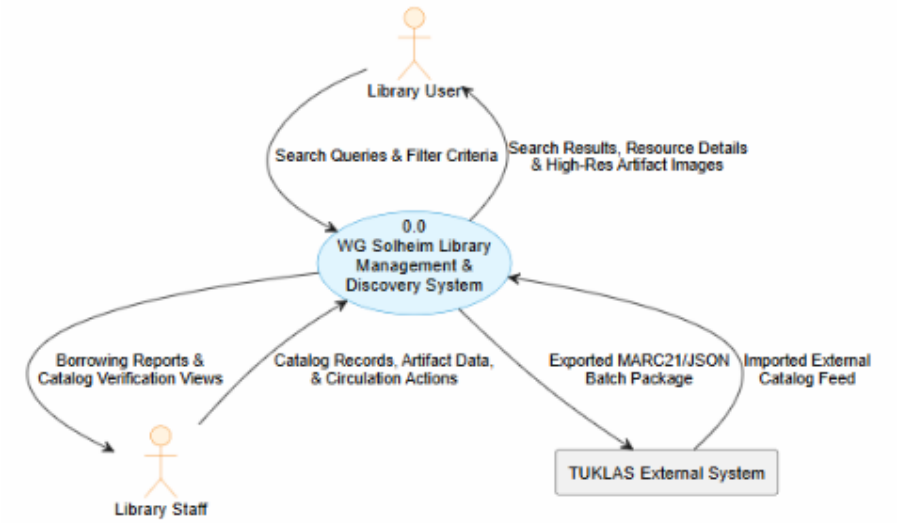
-- ---------------------------------------------------------------------------
-- 5. Artifact images (one artifact can have multiple images)
```

```
-- -----------------------------------------------------------------------------

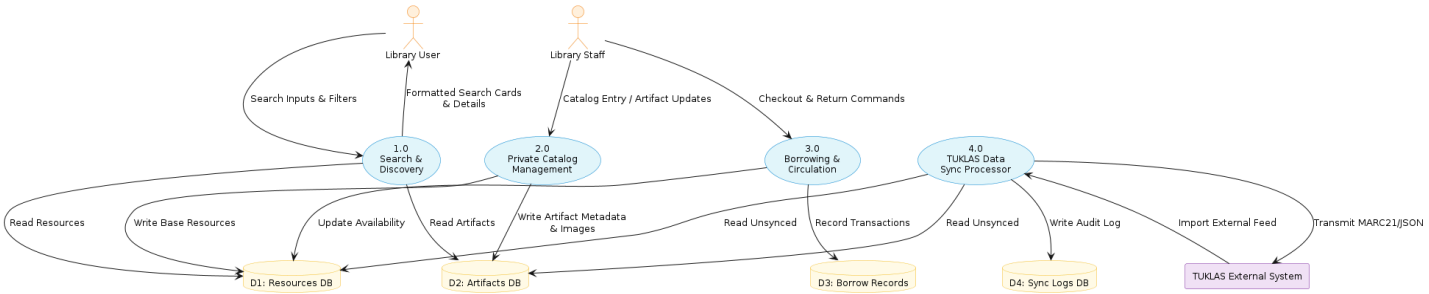
CREATE TABLE IF NOT EXISTS `ARTIFACT_IMAGES` (
    `image_id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    `resource_id` VARCHAR(50) NOT NULL,
    `image_url` VARCHAR(255) NOT NULL,
    `caption` VARCHAR(255) NULL,
    `is_primary` BOOLEAN NOT NULL DEFAULT FALSE,

    CONSTRAINT `pk_artifact_images` PRIMARY KEY (`image_id`),
    CONSTRAINT `fk_artifact_images_artifact`
        FOREIGN KEY (`resource_id`)
        REFERENCES `ARTIFACTS` (`resource_id`)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    INDEX `idx_artifact_images_resource` (`resource_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

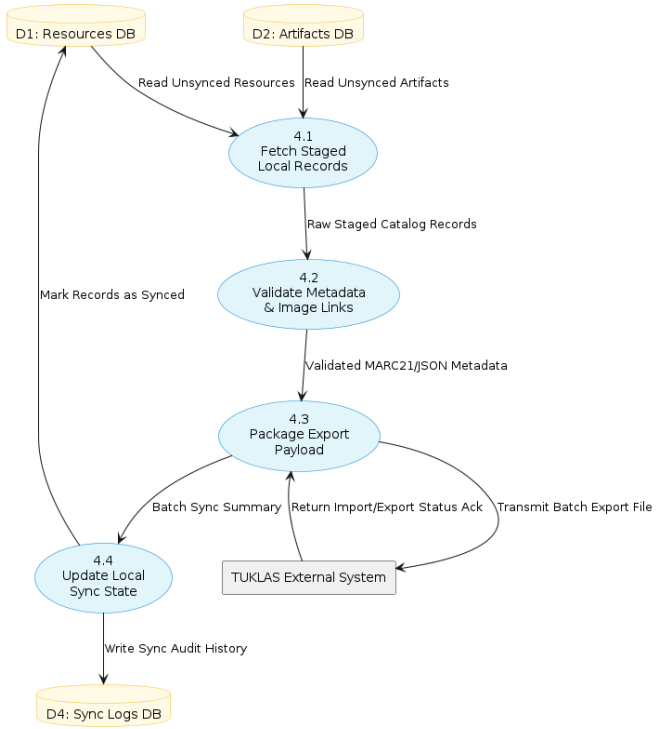
Data Flow Diagrams



DFD Level 0 Diagram



DFD Level 1 Diagram

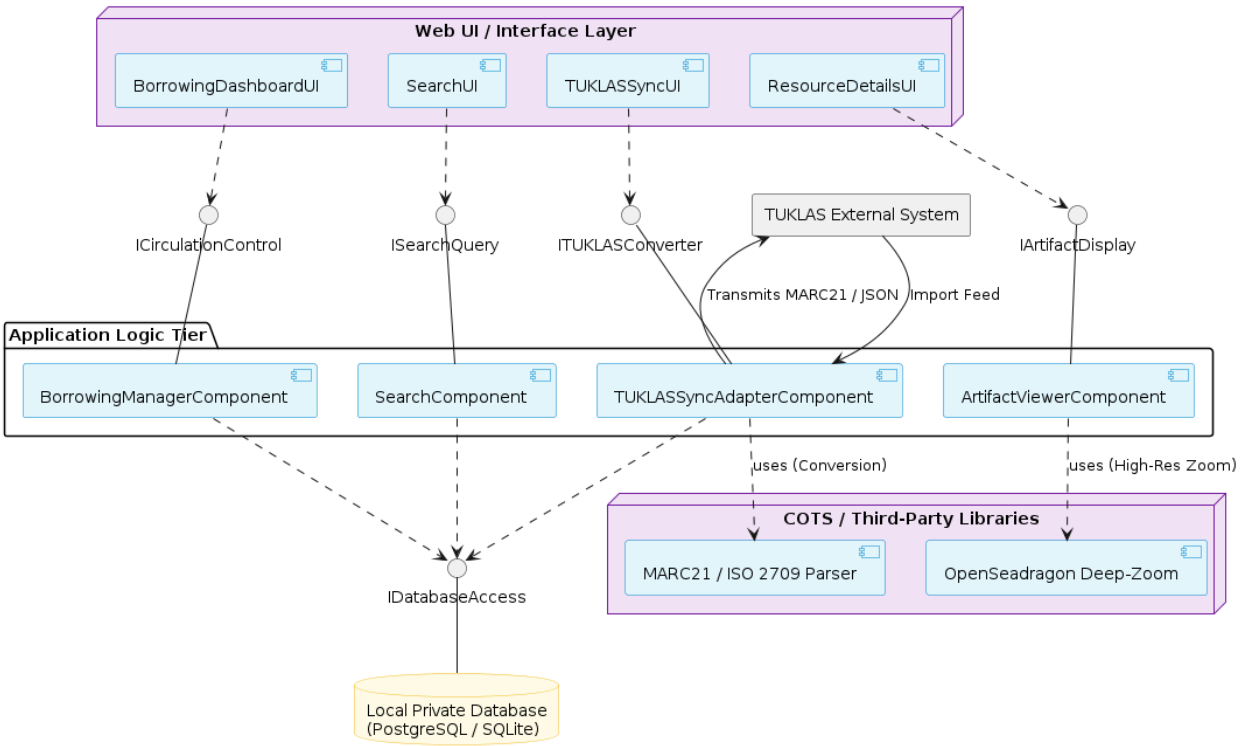


DFD Level 2 Diagram

Component-Level Design

System Components, Interfaces, & Dependencies

The system architecture decomposes the WG Solheim Library System into modular functional components, ensuriang clear component boundaries and explicit interface contracts.



SearchComponent

Search Module Purpose. Manages multi-field lookups across library books, MARC fields, and archaeological collections.

- **Exposed API Interfaces: ISearchQuery**
    - executeSearch(query: String, category: String, filters: FilterMap): List<ResourceCard>
    - getResourceDetails(resourceID: String): ResourceDetailDTO
- **Required System Dependencies: IDatabaseAccess** (queries database entities **RESOURCES**, **BOOKS**, **BOOK_MARC_TAGS**, and **ARTIFACTS**).
- **Internal Architecture Dependencies:** Leverages an inverted index module for efficient in-memory string matching.

ArtifactViewerComponent

Artifact Display Purpose. Controls the visual delivery of historical artifacts, featuring dynamic rendering components for detailed imagery.

- **Exposed API Interfaces: IArtifactDisplay**
    - renderArtifactGallery(resourceID: String): GalleryDTO
    - getSiteContext(systemCode: String): ArchaeologicalContextDTO
- **Third-Party Library Integration:** Embeds OpenSeadragon, an open-source JavaScript deep-zoom viewer, allowing seamless navigation, scaling, and high-resolution examination of artifact pictures without impacting client performance.

BorrowingManagerComponent

Circulation Module Purpose. Powers the staff-facing circulation workflows and borrowing operations.

- **Exposed API Interfaces: ICirculationControl**
    - checkOutItem(userID: String, resourceID: String, dueDate: Date): BorrowReceipt
    - returnItem(borrowID: String, returnDate: Date): CirculationStatus
    - getOverdueList(): List<OverdueRecord>
- **Required System Dependencies: IDatabaseAccess** (updates **BORROW_RECORDS** entries and modifies **RESOURCES.is_available** flags).

TUKLASSyncAdapterComponent

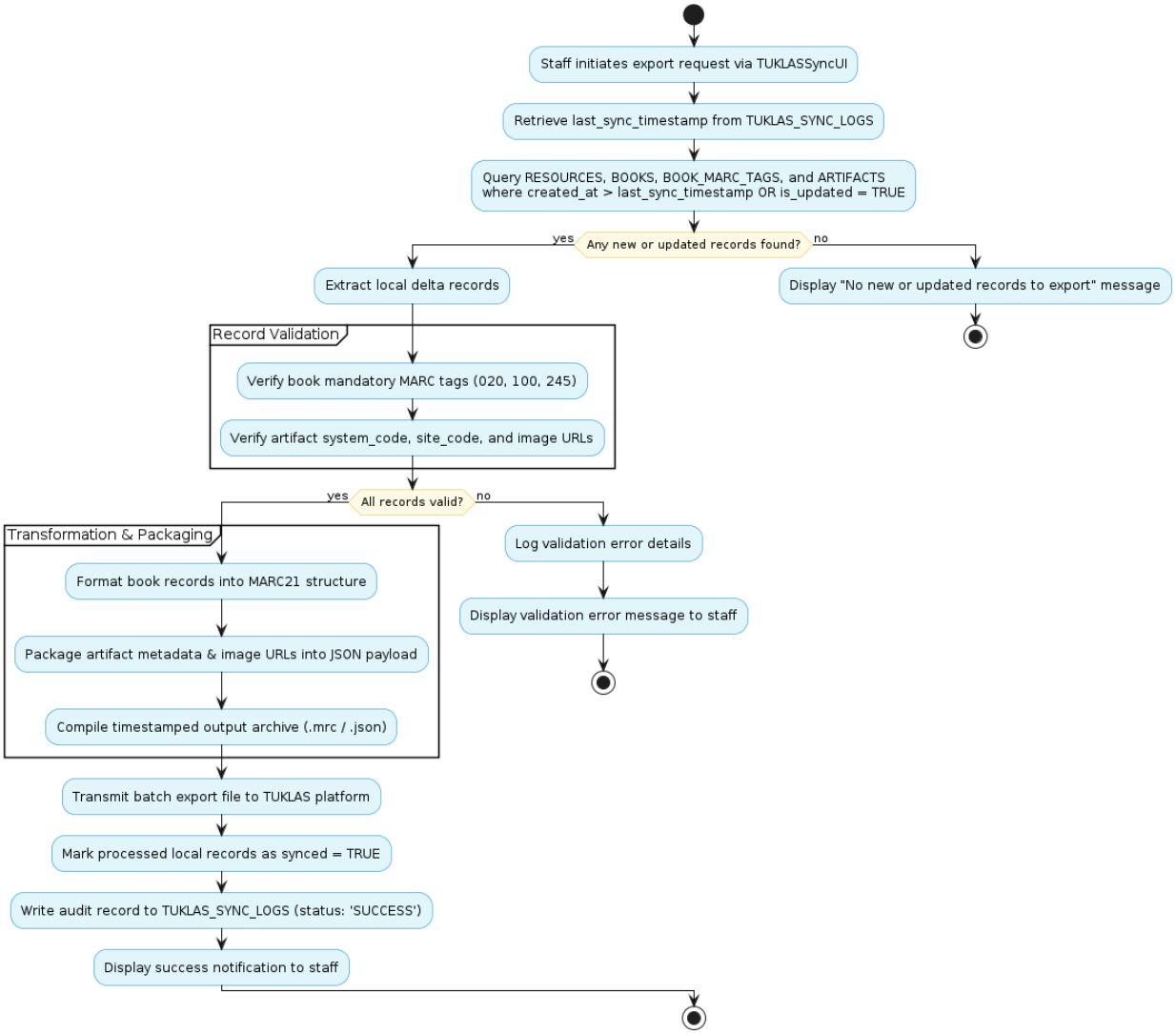
Synchronization Adapter Purpose. Executes dataset transfers between local storage repositories and the centralized TUKLAS platform.

- **Exposed API Interfaces: ITUKLASConverter**
    - exportToMARC21(recordIDs: List<String>): FileStream
    - importFromMARC21(fileStream: FileStream): SyncResult
    - generateJSONBatch(lastSyncTimestamp: Timestamp): JSONPackage
- **Standard Format Processing:** Incorporates standard MARC21 and ISO 2709 serialization utilities to translate local relational entries into standardized bibliographic exchange structures.

Algorithmic & Procedural Detail

Batch TUKLAS Synchronization Algorithm

The following procedure outlines the component logic executed when library staff trigger a batch export to the TUKLAS system:



Local Data Structures & Technical Justification

Inverted Search Index (Hash Table / In-Memory Map)

In-memory key-value structure mapping individual search tokens or keywords to arrays of resource_id references:

$$IndexMap: \{"Manunggul" \rightarrow ["RES - A001"], "Solheim" \rightarrow ["RES - B001", "RES - B004"]\}$$

Prioritizes Query Speed ($O(1)$ lookup time) over RAM consumption. Because library users frequently search and filter by keyword, an in-memory inverted index delivers

instant search suggestions and result cards without burdening the relational database with repetitive SQL LIKE queries**.**

B-Tree Database Indexing

B-Tree / B+ Tree indexes created on primary search keys (call_number, system_code, site_code, isbn, and user_id).

Balances Read Performance against write overhead. Read and lookup operations far outnumber creation and update operations in a library catalog system. Indexing these specific fields ensures near-instant record retrieval during daily search and circulation operations.

Interface Design

WG Solheim Library System serves public library users, circulation staff, cataloging staff, and administrators. Public users search, browse, and place holds; staff catalog records, process loans and returns, and produce reports.

The public navigation has **Catalog, Collections, Artifacts, Services,** and **About**, with persistent search and account access. Staff navigation has **Dashboard, Cataloging, Artifacts, Circulation, Borrowers,** and **Reports**. Facets, result links, record tabs, breadcrumbs, pagination, dashboard cards, and row actions provide context-specific navigation.

The interface workflow follows the standard for web applications: review requirements, sketch layouts, map user goals to actions, define tasks, storyboard screens, and refine the design.

Forms and reports: The charge slip shows borrower and item details, due date, loan rule, staff member, and timestamp. The fine receipt shows its number, borrower, itemized charges, total, payment details, cashier, and timestamp. The borrower card carries the borrower ID and barcode. The monthly circulation report, weekly overdue notice, and monthly acquisitions list are specified for print or web delivery. The draft gives field-level detail for the slip and receipt; periodic-report columns need confirmation.

Interface issues: Provide prompt feedback and progress for longer tasks; field and contextual help; clear, nonjudgmental error messages with recovery steps; and consistent action labels such as “Place hold” and “Print slip.”

Aesthetic Design — Layout and Graphics

Use a responsive grid with consistent placement for navigation, content, and actions. Keep layouts uncluttered, prioritize record and availability information, and adapt the facet rail and

navigation for smaller screens. Use maroon and green accents with neutral backgrounds; pair Playfair Display titles with Inter interface text. Statuses must include words as well as colour. These choices follow content emphasis, whitespace, grouping, and screen size.

Navigation Design

Organize destinations into five navigation semantic units (NSUs): **Find & Discover**, **My Library**, **Collection Management**, **Circulation Desk**, and **Oversight & Reporting**. Each groups the screens and links needed for a user goal.

Scenario	Clicks	Keystrokes	Total	Navigation	Data entry
Search for a title and check availability	2	16	18	18	0
Sign in and place a hold	7	35	42	24	18
Browse artifacts and cite an item	10	0	10	10	0
Catalog a book	9	102	111	12	99
Return an item and settle a fine	7	9	16	7	9
Run and print a monthly report	6	0	6	6	0
Total	**41**	**162**	**203**	**77 (38%)**	**126 (62%)**